Supplementary Material

Anonymous Author(s)

I. Scope and Boundary

The main paper introduces policy intent drift, a normalized browser-enforced policy representation, and minimal semantic conflict witnesses. This supplement expands proof sketches, corpus admission, comparator interpretation, robustness checks, failure accounting, and reuse boundaries. It records the protocol-amended BEP-Deep stress workload used in the main paper. The amendment expands the deterministic workload and algorithmic checks before the upgraded interpretation, while preserving the prior seed fixtures, labels, metrics, and failure policy. It does not reinterpret deterministic fixture results as deployed-site prevalence or claim a browser-engine conformance suite.

The core object remains a witness. A witness relates an admitted public claim, a generated policy surface, a context, a browser-effective judgment, and a contradiction predicate. The claim must be explicit enough to name an expected browser action or a framework precondition that can be connected to such an action. The generated surface must be observable in a response field, framework option, route-level policy, rendering mode, or resource edge. The context contains only variables read by the encoded rules. Rules outside the encoded fragment return unknown and cannot create positive findings.

This boundary is conservative but deliberate. Browser security policy engineering spans specifications, developer documentation, framework defaults, middleware, deployment edges, and browser state. A complete model of all of that behavior would be a different project. The paper instead asks a software-engineering question: when public engineering artifacts make explicit policy claims, can we make the relation among intent, generation, and effective enforcement auditable? The supplement makes that question precise enough for independent audit.

II. Expanded Model

A source-grounded claim records an identifier, family, evidence role, source span, expected action, and applicable rule identifiers. The role separates normative semantics, framework generation, comparator scope, and contextual motivation. A generated surface records its layer, scope, response fields, directives, and generation facts. The layer may be a framework API, middleware default, response field, route handler, or documented generation precondition. A context records document and resource origins, request mode, credentials mode, scheme, state, rendering mode, edge relation, and selected feature. Only variables used by admitted rules are retained. Everything else is left outside the executable fragment.

Normalization is family-specific. Header names are canonicalized case-insensitively, covered directive names are mapped to finite records, known context values are mapped to enumerations, and unsupported syntax is preserved as invalid or unknown rather than silently ignored. The effective judgment is a partial function:

$$\mathsf{Eff}(\mathsf{Norm}(g), c) \in \{\mathsf{allow}, \mathsf{block}, \mathsf{monitor}, \mathsf{clear}, \mathsf{unknown}\}.$$

The action vocabulary is intentionally small. It is large enough to distinguish enforcing, monitoring, sharing, state clearing, and unknown behavior, but small enough to support deterministic proof obligations.

Policy intent drift is a contradiction relation. For an admitted claim i, generated surface g, and context c, a witness exists only when $\mathsf{Eff}(\mathsf{Norm}(g), c) \neq \mathsf{unknown}$ and the returned action contradicts $\mathsf{Exp}(i)$ under the family-specific predicate. A single total lattice over all browser policies would hide important distinctions: blocking a script edge, sharing a CORS response, storing HSTS state, requiring resource opt-in, and disabling a feature are different browser actions. BEP-IR therefore keeps family-specific judgments behind a common witness interface.

The table is not a taxonomy of headers. It is a list of semantic hinges that must survive minimization. A report-only witness is no longer the same witness if the delivery disposition is deleted. A CORS witness is no longer the same witness if credentials mode is removed. An embedding witness is no longer auditable if the resource edge disappears. This is why context is part of the witness object.

TABLE I
Encoded mechanisms and witness hinges.

Mechanism	Required semantic hinge
CSP delivery	report-only disposition does not imply blocking
CSP fallback	effective directive selection for a resource edge
CSP generation	nonce freshness depends on rendering state
CORS	ACAO and ACAC interpreted with credentials and origin relation
HSTS	secure delivery and max-age update or clearing
Embedding	document policy plus resource opt-in and request mode
Isolation	covered opener, embedder, and resource preconditions
Permissions	selected feature and selected requesting origin
Layer composition	ordered generation surfaces and conjunctive CSP intersection
Repair synthesis	counterfactual edit must remove target issue under the same oracle

III. Proof Sketches

Theorem 1 (Normalization determinism)**.** *For every finite covered observation, normalization returns a unique normal form or* unknown*.*

Sketch. Each family parser is a deterministic map from finite input records to finite normal forms. Header canonicalization is deterministic. Directive precedence is fixed before evaluation. Malformed tokens produce explicit invalid or unknown terms. Duplicate handling follows the locked rule recorded in the study protocol. Because the normalization pipeline is a composition of deterministic finite functions, the result is unique.

Theorem 2 (Effective-judgment determinism)**.** *For a normalized covered object and a locked context,* Eff *returns at most one action.*

Sketch. The proof is by family case analysis. CSP dispatches first on delivery disposition and then on effective directive selection. Report-only disposition exits through the monitoring branch for covered violations. CORS dispatches on a finite tuple of credentials mode, ACAO category, ACAC state, and origin relation. HSTS dispatches on transport scheme and max-age state transition. Embedding, isolation, and Permissions-Policy fragments are finite predicate combinations with disjoint guards. Unmatched guards return unknown.

Lemma 1 (Report-only non-blocking)**.** *In the encoded CSP fragment, report-only delivery can yield* monitor *for a covered violation but cannot yield* block*.*

Sketch. Delivery disposition is checked before directive content is interpreted as enforcement. The report-only branch records the covered violation as monitoring and does not enter the enforcing branch. The lemma supports witnesses in which the policy text looks restrictive but the research surface contradicts a blocking claim.

Theorem 3 (Witness soundness)**.** *Every emitted positive witness contains an admitted claim, a locked generated surface and context, a non-unknown effective judgment, and a rule-level contradiction with the claim's expected action.*

Sketch. The generator iterates only over admitted claims and locked fixtures. It normalizes the surface, evaluates the effective judgment, rejects unknown, and then applies the family-specific contradiction predicate. A witness record is created only after all checks pass. The theorem is conditional on the encoded fragment; it does not assert completeness for unmodeled browser behavior or exploitability.

Theorem 4 (Negative-control preservation)**.** *A locked negative control cannot emit a positive witness unless the control label, expected action, or semantic rule violates the validation invariants.*

Sketch. Negative controls have expected issue label `none` and are paired with the mechanism they test. The generator emits positives only through contradiction predicates. For each paired control, the generated surface and context satisfy the non-conflicting side of the rule or return no contradiction. Baseline outputs are never inputs to the semantic oracle, so they cannot create positives.

TABLE II
Denominator roles.

Role	Evaluation contribution
Fixture-backed claim	can instantiate positive or negative obligations
Context claim	grounds model scope or generation precondition
Baseline claim	grounds comparator scope only
Excluded claim	omitted from executable denominator

Theorem 5 (One-deletion minimizer preservation)**.** *If the minimizer returns witness w' for issue r, then w' still triggers r, and no declared single deletion can be applied to w' while preserving r.*

Sketch. The minimizer considers a finite set of deletion operators over headers, directives, source tokens, and context fields. A deletion is committed only when re-evaluation under the same oracle emits the same issue class for the same claim. Each committed deletion decreases a finite unit count, so the loop terminates. At termination every declared one-step deletion has been tried and rejected. The guarantee is one-deletion minimality, not global shortest-string minimality.

The proof obligations also explain the conservative use of unknown. Unknown is not a weak positive and not a weak negative. It is an explicit refusal to decide outside the encoded fragment. This reduces apparent coverage, but it protects the meaning of positives. A positive is a contradiction under a declared rule; an unknown is a boundary note.

IV. Corpus Admission and Validation

A public claim is admitted into the executable denominator only when it states an observable browser-policy effect or a framework generation surface that can be connected to such an effect. General hardening advice, marketing text, inferred organizational intent, and statements requiring private application context are excluded from the positive denominator. They may remain contextual evidence, but they do not create labels.

The validation procedure is deterministic. It checks that claim identifiers are unique, source spans are present, family and role fields are populated, rule identifiers exist, fixture identifiers resolve, expected labels are consistent, negative controls are marked as such, and checksums are stable. A separate claim-coverage audit assigns every admitted claim to one of three roles: fixture-backed evidence, context-surface evidence, or baseline-scope evidence. An admitted-source/source-span closure audit then checks that each admitted claim has exactly one source-span ledger row and that the row resolves through the same admitted-source snapshot used by materialization. A negative-control certificate audit then checks every ordinary and paired-repair control under both the operational oracle and the independent decision table.

TABLE III
LOCKED VALIDATION SUMMARY.

Validation object	Count
Source-grounded claims	45
Fixture-backed claims	26
Context or baseline claims	19
Semantic rules	35
Claim, source, and source-span closure	45 / 45
BEP-Deep deterministic fixtures	972
Expected-positive fixtures	418
Negative controls	554
Negative-control certificates	554 / 554
Blocking validation failures	0

TABLE IV
COMPARATOR INTERPRETATION.

Comparator state	Meaning
Agreement	applicable comparator flags the same scoped conflict
Disagreement	applicable comparator output differs
Not applicable	fixture is outside declared tool scope
Unavailable	public tool cannot run unmodified locally
Unsupported	fixture cannot be represented by tool input

Negative controls are paired with mechanisms where possible. CSP controls use enforcing delivery or an explicit directive that matches the claim. CORS controls remove credentials or use a serialized authorized origin. HSTS controls use secure delivery or a positive age. Embedding controls supply compatible resource opt-in. Permissions controls include the selected requesting origin. These controls are more demanding than arbitrary benign examples because they test the boundary at which the oracle should stop emitting.

V. ALGORITHMS AND COMPARATOR PROTOCOL

The witness generator lowers admitted claims into scoped obligations, evaluates each obligation under the effective-judgment relation, and emits a witness only for a non-unknown contradiction. The minimizer then deletes declared units while preserving the same target issue. It does not synthesize new policies, execute applications, reorder deployment layers, or search arbitrary strings. Its role is explanation, not repair synthesis.

Comparator rows are interpreted by declared task scope. A CSP-specific evaluator is compared only on CSP-applicable fixtures and only for its inspection surface. A header scanner is compared only when the obligation can be represented as response-header presence or absence. A preload-style HSTS comparator is scoped to eligibility-style questions and is not treated as the ground truth for every HSTS state transition. Non-applicability is a semantic-scope outcome. Unavailability is an artifact-state outcome. Neither is silently converted into an internal result.

This protocol avoids two misleading readings. First, a tool is not counted as wrong merely because the paper asks a broader question than the tool's declared task. Second, the paper does not replace unavailable external logic with project logic and then label the result as an external baseline. The RQ3 result is therefore a scope result: common inspection surfaces preserve some evidence needed by intent-drift witnesses and discard other evidence such as rendering state, request credentials, stored state, or resource opt-in.

TABLE V
FINITE-STATE CORE VERIFICATION.

Verification object	Count
Invariants	13
Abstract states	165
Failures	0
CSP non-expansion checks	75
CORS share/cache checks	37
HSTS state/scope checks	15
Embedding/permission/layer checks	28

VI. FINITE-STATE SEMANTIC-CORE VERIFICATION

The main paper states proof obligations for the encoded fragment. We also execute them as finite-state checks over abstract CSP, CORS, HSTS, embedding, permissions, and layer-composition domains. The verifier checks 13 invariants over 165 states and reports zero failures. The check is not a theorem prover for the complete web platform; it is an executable bridge between the proof skeleton and implementation.

VII. INDEPENDENT ORACLE AND MUTATION ADEQUACY

The validation layer includes a second oracle written as decision tables rather than as the operational witness generator. It shares the same admitted fixture input, but it re-derives each covered issue through a separate rule table over delivery disposition, directive selection, request credentials, cache variation, HSTS state and scope, embedding/resource opt-in, isolation headers, feature allowlists, and ordered layer composition. Agreement is checked over all 972 BEP-Deep fixtures and over 351 generated finite states. Both checks report zero mismatches. A separate cross-policy contract verifier checks 7 source-level composition obligations over 111 finite states, including the rule that a CORS header authorizes a COEP resource edge only in a covered cors-mode request. This is not a theorem about the web platform; it is a redundancy check that reduces the chance that a programming error in the generator is silently confirmed by labels generated through the same code path.

A source-level directive audit separately checks 6/6 CSP fallback micro-obligations, including the script-src-elem, script-src, default-src ordering. A finite theorem kernel then enumerates 30 semantic theorems over 33,513 states, covering CSP meet and directive specificity, CORS credential and cache obligations, HSTS state partitions, COEP request-mode separation, CORP scope, Permissions-Policy allowlists, and ordered generation layers. The mutation adequacy audit changes one semantic hinge at a time. Weakening mutants remove

necessary checks, such as treating report-only CSP as enforcement, ignoring default-src fallback, allowing credentialed wildcard CORS, honoring HSTS over HTTP, or treating COEP as independent of resource opt-in. Over-reporting mutants approximate checklist-style mistakes by flagging conflicts whenever a header family is merely present. BEP-Deep kills 28/28 rule-level mutants: 22 by positive obligations and 6 by negative controls, including paired repair controls. A second obligation-level mutation farm kills 600/600 omission, rule-confusion, family-overgeneralization, intent-overgeneralization, negative-control-injection, and class-confusion mutants by concrete locked fixture mismatches rather than by aggregate result counts. These numbers should not be read as a general mutation score for arbitrary browser-policy semantics. They are adequacy checks for the encoded rule families used in the paper.

The proof-carrying witness certificate checker adds a row-local validation layer for every emitted positive witness. Each certificate records the fixture identifier, issue label, source-claim identifiers, semantic rule identifiers, independent decision-table result, paired repair control, minimality result, and repair-obligation result. A certificate is accepted only when all these obligations hold. The checker verifies 418/418 positive certificates, including the HSTS preload family. The companion negative-control certificate checker verifies 554/554 controls: every ordinary and paired-repair control is clean under both oracles, has a stable hash, resolves to admitted source claims, and, for paired controls, names the positive fixture and target issue it neutralizes. A release adversarial validation layer, BEP-Max, surrounds the locked denominator with 4,306 generated cases: 3,888 semantic-preserving surface variants and 418 positive-preserving near-repair variants. All cases preserve the expected result under both the operational oracle and the independent decision table. A companion integrity audit verifies 4,306/4,306 fresh content hashes, unique case identifiers, valid source links, and label-preserving variants. A repair-frontier checker validates 418/418 paired repairs over 1,230 local candidates, and a boundary-coverage audit confirms that all 25 issue classes have positive witnesses, controls, certificates, repair frontiers, and adversarial variants.

TABLE VI
ORACLE-INDEPENDENCE AND ADEQUACY CHECKS.

Check	Result
Locked fixture agreements	972 / 972
Generated finite-state mismatches	0 / 351
Directive-fallback conformance	6 / 6
Semantic mutants killed	28 / 28
Obligation-level mutation farm	600 / 600
Finite theorem kernel	30 / 33,513 states
Proof-carrying positive certificates	418 / 418
Negative-control certificates	554 / 554
BEP-Max cases / integrity	4,306 / 4,306
Repair frontiers	418 / 418
Issue-class validation ladders	25 / 25
Mutants killed by negative controls	6
Rules resolving to public source ids	35 / 35

TABLE VII
POSITIVE WITNESSES BY SEMANTIC CONFLICT CLASS.

Semantic conflict class	Count
CSP report-only not enforced	6
CSP fallback/source allowance conflict	32
Nonce CSP/static rendering incompatibility	24
Credentialed CORS wildcard blocked	24
CORS ACAC case sensitivity	6
Duplicate ACAO not shareable	6
Dynamic origin missing Vary	24
Dynamic origin without Vary	6
Reflected origin with credentials	24
HSTS ignored over insecure transport	24
Invalid HSTS max-age ignored	6
HSTS max-age zero clears state	6
HSTS missing includeSubDomains	6
HSTS preload criteria not met	6
HSTS subdomain scope not covered	24
COEP require-corp blocks no-cors resource	24
CORP same-site scope mismatch	6
Cross-origin isolation incomplete	24
CSP frame-ancestors meta unsupported	6
CSP frame-ancestors report-only	6
Conjunctive CSP blocks required script	32
Multiple CSP policy overblocks trusted script	6
Layered override drops enforcement	24
Permissions-Policy feature disabled	24
Permissions-Policy feature overallowed	6

VIII. ADDITIONAL EVALUATION DETAIL

The positive denominator is balanced by semantic mechanism. The counts are not prevalence estimates. They are designed to exercise delivery, fallback, generation, request context, browser state, cross-policy embedding, isolation preconditions, and feature allowlists under a locked fixture denominator. The table reports the frozen 418-positive BEP-Deep denominator, not a seed-only subset.

Ablations remove semantic components while preserving denominator and labels. Losses are dependency checks: removing CORS credentials should remove CORS credential witnesses; removing HSTS state should remove HSTS transition witnesses; removing the minimizer should not remove witnesses but should affect reduction. This interpretation matters because the ablation table is not a performance tuning table. It is a test of whether the semantic mechanisms that define drift are actually necessary for the emitted witnesses.

Robustness checks perturb irrelevant material while preserving the locked denominator. A report-only witness should remain stable under unrelated response fields and should change when delivery disposition is repaired. A fallback witness should tolerate irrelevant directives and should change when the effective directive is repaired. A CORS witness should be sensitive to credentials mode and origin relation, not to unrelated fields. HSTS witnesses should be sensitive to scheme, max-age, and stored state. Embedding witnesses should be sensitive to document policy, resource opt-in, request mode, and origin relation.

Scalability checks use deterministic replication of the locked

TABLE VIII
ABLATION DEPENDENCY SUMMARY. LARGER LOSS MEANS MORE POSITIVES DEPEND ON THE REMOVED COMPONENT; BOLD MARKS THE LARGEST LOSS AND UNDERLINE MARKS THE SECOND LARGEST.

Removed component	Lost positives
Layered policy composition	**62**
CORS credentials/shareability	<u>60</u>
HSTS state transition	54
HSTS scope/preload context	36
CSP fallback/source list	32
CORS cache context	30
Permissions allowlist	30
CSP delivery disposition	24
Rendering context for nonce CSP	24
COEP/CORP/CORS embedding	24
COOP/COEP isolation edge	24
CSP framing delivery	12
CORP scope semantics	6

TABLE IX
REPRESENTATIVE MINIMIZED WITNESS SHAPES.

Class	Essential retained evidence
CSP report-only	monitoring delivery and covered blocking claim
CSP fallback	selected effective directive and edge
Nonce/static	nonce claim and rendering state
CORS wildcard	credentials mode, ACAO, ACAC
Reflected CORS	reflected origin and restrictive intent
HSTS insecure	transport scheme and state update claim
HSTS clear	max-age zero and prior known state
Embedding	document policy plus resource opt-in edge
Permissions	selected feature and origin relation

corpus. Replication is not new evidence and does not increase the denominator. It verifies CPU-native execution and guards against accidental superlinear behavior at the locked scale. The largest run in the main paper uses replicated fixtures to check local execution without GPU acceleration, live traffic, private accounts, or commercial services.

IX. WITNESS SHAPES AND AUDIT PROCEDURES

A minimal witness is an explanation unit. It removes irrelevant headers, directives, tokens, and context fields while preserving the same issue class and claim. The result is not merely shorter. It identifies the repair site: delivery disposition, directive selection, rendering state, request credentials, transport state, resource opt-in, isolation precondition, or feature allowlist.

An auditor can inspect a report-only witness by checking four facts: the claim expects blocking, the generated field is monitoring disposition, the context contains a covered resource edge, and the effective rule returns monitor rather than block. A fallback witness is audited by following directive selection for the resource edge named by the claim. A nonce/static witness is audited as a generation precondition, not as a browser CSP parser bug. A CORS witness is audited by checking credentials mode, ACAO category, ACAC state, and origin relation together. An HSTS witness is audited as a state transition. An embedding witness is audited as a document-resource edge.

The audit procedure makes disagreement local. If an auditor disputes the source interpretation, the disagreement concerns one claim span. If an auditor disputes the generated surface, the disagreement concerns one framework or policy object. If an auditor disputes the effective action, the disagreement concerns one semantic rule. This locality is the main advantage of a witness over a scanner grade.

X. FAILURE ACCOUNTING AND EXTENSION PROTOCOL

The locked protocol distinguishes editorial changes from experimental amendments. Prose rewrites, figure drawing, and bibliography pruning do not alter the experiment. Adding a source claim, deleting a fixture, changing an expected issue label, changing a negative-control role, changing a baseline scope, or changing an ablation component would alter the reported experiment and require a new lock. This discipline is important because a semantic benchmark can otherwise be improved after results are known by adding favorable examples or deleting inconvenient failures.

Failures are counted where they occur. A fixture parse failure would remain in the denominator and would be reported as a processing failure. A semantic unknown on an expected-positive fixture would be reported as unsupported rather than converted to a negative. A negative-control violation would reduce negative-control cleanliness. A baseline wrapper failure records canonical availability, unsupported, excluded, or error status; it does not justify replacing the baseline with project logic. A baseline-contract audit checks that the BEP-Deep fixture-level external-baseline probe exists and that external baseline rows are not remapped into semantic confusion counts without a declared oracle mapping. A minimization no-op leaves the witness valid but changes the reduction metric.

A new source family should be admitted in four steps. First, the researcher records explicit public evidence and separates normative semantics from generation advice and comparator documentation. Second, the researcher defines a partial judgment with an unknown boundary. Third, the researcher constructs paired positive and negative fixtures before observing algorithmic results. Fourth, the researcher adds an ablation that removes the new rule family and checks that only dependent positives disappear.

A new framework family should be admitted differently. The first question is not whether the framework is popular, but whether it exposes an observable generation surface: a default header, middleware option, route-scoped override, rendering-state precondition, or migration mode. General advice without an observable generated surface can remain contextual evidence but should not become a fixture-backed positive.

XI. THREATS AND REUSE

Construct validity is bounded by the definition of policy intent drift. The measured outcome is not exploitability, severity, developer belief, or deployed prevalence. It is a contradiction between an explicit expected browser action and

an encoded effective judgment in a covered context. Internal validity threats include rule encoding mistakes, fixture-construction mistakes, and comparator mapping mistakes. The study mitigates them with deterministic validation, paired negative controls, ablations, robustness checks, proof obligations, and transparent baseline-status records. The model is not mechanized in a proof assistant.

External validity is bounded by the deterministic corpus. The corpus is heterogeneous but not representative of the web, repositories, or framework applications. A deployed-site measurement would require a sampling frame, authorization boundary, rate limits, server-state policy, regional controls, temporal controls, and false-positive adjudication. A developer-workflow study would require human-subjects or public-interaction methodology. A browser-conformance study would require browser-engine test harnesses and versioned engines. The present paper supplies a semantic unit that those studies could use, not the studies themselves.

The reusable unit is the witness object. It records a claim, a generated surface, a context, an effective judgment, and a rule identifier. A future tool can challenge the rule, add a family, mechanize a fragment, or run the object as a regression test. That is different from preserving a scanner grade or a contemporary framework default as ground truth. The witness keeps the auditable relation, which is the technical object the main paper asks readers to evaluate.

XII. Additional Proof Detail by Mechanism

This section expands the mechanism-level proof obligations used by the main paper. The purpose is not to mechanize the entire web platform. It is to show that the executable fragments used in the locked experiment have stable proof boundaries and that each reported witness follows from a finite, auditable rule set.

a) CSP delivery and fallback.: For covered CSP fixtures, normalization first classifies delivery as enforced or report-only. Delivery classification is deterministic because it is a function of a finite header-name equivalence class. The effective directive relation then selects the most specific covered directive for the requested edge, falling back only to the encoded default directive when the specific directive is absent. If the delivery is report-only, the returned action is monitor even when the selected directive would otherwise block. Thus a report-only witness is sound when the admitted claim requires blocking and the generated surface is monitoring-only. A fallback witness is sound when the selected effective directive permits an edge that the admitted claim expects to block.

b) CORS shareability.: The CORS fragment is not a general model of networking. It is a shareability judgment over the response fields, request origin, credentials mode, and the Access-Control fields covered by the locked rules. The wildcard credential case follows because the rule for credentialed sharing requires an origin-specific allowance rather than a wildcard. The reflected-origin case is classified separately because it can satisfy a syntactic origin check while contradicting an admitted restrictive-origin claim. The two classes are intentionally not merged: one is a browser-level rejection, the other is an over-broad generation relation.

c) HSTS state.: The HSTS fragment is a state-transition judgment. A header received on an insecure scheme leaves the known-host state unchanged, and a secure max-age zero update clears the state. The judgment is deterministic because, for the covered inputs, scheme classification and max-age parsing choose a single transition. This is why HSTS fixtures are not evaluated as simple header-presence examples. A positive state claim can fail even when a syntactically plausible header is present, and a clearing claim can be correct even when a scanner treats the field as a security-header occurrence.

d) Embedding and isolation.: COEP, CORP, CORS, and COOP fixtures are evaluated as edge judgments. A document policy alone is insufficient for a positive isolation judgment when the resource edge lacks the required resource-side opt-in or compatible CORS mode. The proof obligation therefore has two parts: the document-side precondition and the resource-edge precondition. A witness is emitted only when both the source claim and the generated surface make the relevant edge auditable.

e) Permissions.: The Permissions-Policy fragment is an allowlist judgment over selected features and origins. It is conservative: unknown features and unsupported syntax do not produce positive findings. For covered features, the disabling witness is sound when the selected requesting origin is outside the effective allowlist and the admitted claim expects the feature to remain enabled for that origin.

XIII. Coding Adjudication and Traceability

The empirical layer is deterministic source traceability, not a human-subjects coding study. Each admitted claim row records a source unit, a source role, a family, a paraphrased explicit claim, a rule reference, and a fixture relation. Validation checks three invariants. First, every fixture-backed claim must point to at least one locked fixture. Second, every fixture must point to an admitted rule family and an expected label. Third, every positive label must have a source-backed expected action rather than an inferred author intention.

This design prevents two common sources of evaluation drift. The first is denominator drift: after observing results, a researcher might decide that a difficult source claim was merely contextual. The locked protocol prevents this by separating fixture-backed, contextual, and baseline claims before execution. The second is interpretation drift: a broad documentation sentence might be converted into a stronger security claim than it actually makes. The protocol prevents this by requiring the expected action to be explicit enough to name a covered browser-effective judgment. When a source describes only an operational caveat, such as a framework rendering mode or a comparator's input scope, the claim can shape context but cannot become an expected-positive semantic conflict.

Adjudication is therefore row-local. A disputed row can be repaired by changing its role, removing it from the executable denominator, or adding a protocol amendment before experiments. It cannot be silently reinterpreted after results are known. The locked experiment uses this discipline to keep 45 admitted claims, 35 semantic rules, 116 base fixtures, and 972 BEP-Deep deterministic fixtures fixed for the upgraded run. The BEP-Deep denominator contains 418 expected-positive conflicts and 554 negative controls; earlier seed results remain traceable but are not the main-paper denominator.

XIV. Comparator and Baseline Scope

The comparator design is deliberately conservative. A comparator is included only for its declared task. CSP Evaluator is used on CSP-specific inputs because its documented scope is CSP analysis and bypass-oriented feedback. Header-presence controls are used as conservative controls because they model a common engineering shortcut: treating the presence of a field as evidence of protection. HSTS-preload criteria are reported as a documented criterion control, not as a complete HSTS deployment oracle. Baselines that are unavailable in the current clean environment are reported as unavailable rather than replaced with project logic.

This policy affects interpretation. A disagreement is not automatically a false positive or false negative by another tool. It is evidence that the comparator's declared scope does not cover an intent-drift obligation. For example, a report-only CSP may be syntactically meaningful to a CSP checker, but the intent-drift question asks whether the source claim expected blocking. A header-presence control can observe an HSTS field while missing the secure-delivery state transition. A preload criterion can be meaningful for preload-list eligibility while remaining orthogonal to a route-local or state-clearing witness. The main paper therefore reports disagreement by semantic feature rather than as a leaderboard.

XV. Expanded Robustness and Scalability Interpretation

Robustness variants check whether witness generation depends on incidental string formatting. BEP-SpecBench now contains 4,180 source-derived boundary cases over 29 rule aliases outside the locked denominator, including header-case normalization, header-order perturbation, benign padding, irrelevant-header stability, Vary no-op controls, reporting/cache no-ops, and context-noise variants. The metamorphic audit preserves issue class, claim identity, and expected label while changing nonessential order, capitalization, duplicate neutral fields, and context fields outside the rule dependency set, yielding 3,954 checked obligations. BEP-Max expands the locked denominator into 4,306 adversarial checks: header-name case, surface order, irrelevant response fields, policy whitespace, and plausible but ineffective near-repairs. Each generated case is content-addressed after mutation, linked back to its source fixture, and checked by both oracles. The expected property is not that every syntactic mutation remains equivalent in the full web platform. The expected property is that covered normalizations remove irrelevant variation, that near-repairs do not erase true positives, and that repaired controls remain clean. The release protocol span is A001–A099. The artifact adds 9,720 required shadow replays, 4,860 identifier-blind replays that erase benchmark metadata, 2,916 whole-corpus neutral stability replays, 546 causal counterfactual activations from clean controls, 418 repair-delta replays, 418 auditor-facing evidence-card objects, 9,586 benchmark-adjacent disjointness rows, 48,600 deterministic scale-stress cases, 78 pure decision-function checks, a 101-command reproduction ladder with 101 commands, 111 release-validation layers, and 30 proof cards accounting for the finite theorem kernel, 546 counterfactual round-trips, 5,152 oracle explanation-equivalence cases, 280 rule trace-matrix obligations, 14 threat-closure rows, 418 evidence-path multiplicity checks, 225 minimal-pair obligations, five deterministic stratification folds, a process-trace hygiene audit, stale numeric surface scanning, review-criteria closure, criteria-to-evidence closure, contribution trace auditing, reference role-balance auditing, oracle structural independence, deliverable trio readiness, paper rhetoric auditing, repository-entrypoint auditing, overclaim-boundary scanning, compiled-PDF text-surface auditing, public provenance and data-rights closure, repository upload readiness, and a release scorecard.

Scalability replication is also intentionally modest in claim. Replication does not create new evidence. It stresses deterministic parsing, normalization, witness generation, minimization, and output summarization under CPU-native workloads. The largest locked run repeats the fixture set without changing its labels. The purpose is to show that the method is not tied to interactive inspection or manual debugging. It is not evidence of live-web prevalence, production vulnerability rate, or browser-market behavior.

The ablation interpretation follows the same logic. Removing a semantic component should remove only the positives that depend on that component. If removing HSTS state affected CSP fallback, the model would be entangled. If removing rendering context affected CORS wildcard sharing, the witness interface would be too coarse. The locked ablations therefore test architectural separation as much as detection coverage.

XVI. Additional Witness Audit Examples

a) Report-only CSP.: The admitted claim expects blocking for a script edge. The generated surface is delivered through the monitoring header rather than the enforcing header. The normalized context identifies a covered script request. The effective action is monitor, so the contradiction is between an expected block and an effective monitoring action. The minimized witness keeps only delivery disposition, the selected directive, the edge, and the source claim.

b) Nonce with static rendering.: The admitted claim expects nonce-based script authorization. The generated surface is associated with a static or cached rendering state in which a per-request nonce cannot be freshly bound to each response. The witness is not a browser parser error. It is a

generation-precondition conflict: the framework-visible path cannot supply the dynamic value assumed by the policy claim.

c) Credentialed wildcard CORS.: The admitted claim expects a credentialed caller to access an API response. The generated surface contains wildcard ACAO with credentials enabled. The effective shareability judgment blocks credentialed wildcard sharing in the covered fragment. The minimized witness keeps credentials mode, request origin category, ACAO category, ACAC state, and the expected sharing claim.

d) HSTS over insecure transport.: The admitted claim expects HSTS state to be established. The generated response includes an HSTS field, but the context scheme is insecure. The state-transition rule leaves the known-host state unchanged. The witness keeps the scheme, max-age field, prior state, and expected establishment claim. A header-presence explanation would miss the contradiction.

e) Cross-origin embedding.: The admitted claim expects isolation. The generated surface contains document-side policy fields, but an embedded cross-origin no-cors resource lacks a compatible resource-side opt-in. The effective edge judgment blocks or prevents the expected isolation condition. The witness keeps only the document policy, resource origin relation, request mode, and resource-side opt-in state.

XVII. Reuse Boundary

The evaluation package is intended for deterministic reproduction and extension, not for unsupervised assessment of arbitrary deployed sites. A new user can reuse the source-admission schema, the BEP-IR witness interface, the minimization criterion, the ablation template, and the conservative comparator policy. A new user should not treat the locked fixture counts as a measurement of the public web, should not infer unsupported browser policies from unknown judgments, and should not replace unavailable external tools with lookalike internal code while reporting them as baselines.

A valid extension should preserve three locks. The evidence lock records what public claim was used. The semantic lock records which variables a rule reads and when it returns unknown. The experiment lock records which fixtures, labels, negative controls, ablations, robustness checks, and stopping conditions were fixed before execution. These locks are more important than any single implementation detail because they make the paper's central claim reproducible: policy intent drift can be evaluated as a relation among public intent, generated policy surface, and browser-effective judgment.

XVIII. Supplement Closure

This supplement expands the obligations compressed by the ten-page main paper. It clarifies that the model is partial, unknown is conservative, validation is deterministic rather than human-coded, proof claims are fragment-scoped, comparator results are scoped to each comparator task, and fixture results are not prevalence estimates. Those details are not caveats appended after the fact. They are what make a minimal semantic conflict witness auditable.

The supplement should be read witness-first. Start with an explicit source-grounded claim, inspect the generated surface and context, evaluate the covered browser-effective judgment, confirm the contradiction, and then inspect the minimized explanation and paired control. That path is the paper's central technical hook: browser-enforced policy engineering can be evaluated as a relation among intent, generation, and effect, rather than as a checklist of headers.